Name: Arne Lieten

Email: Arne.Lieten@vub.be

Student Number: 0642644

VRIJE UNIVERSITEIT BRUSSEL

Pedigree Dog App: Phase 1

Table of Contents

Table of Contents 1
Introduction 1
Scope and architecture 1
Common ADTs 3
- **dog-profile** 3

Server ADTs 3
- **in-memory-database** 3
- **database-handler** 4
- **process-orchestrator** 4

Client ADTs 4
- **client-orchestrator** 4
- **main-window** 5
- **create-profile-screen** 5
- **main** 5

Testing 6

Introduction

This design document outlines the phase 1 design of a pedigree dog matching application. The idea is to match pedigree dogs based on their age, hobby and type of relation. Later in the project more things can be taken into account such as their family tree, location, among other factors. This design document describes the main ADTs and their dependencies, as well as some pointers on testing.

Scope and architecture

Owners create profiles for their pedigree dogs. Phase 1 provides four independent GUIs and one shared server that stores profiles in memory. Each profile contains a name, age, hobbies and type of relation. Additional fields are left out for now. The application architecture is shown in Image 1. The application has a clear separation between client components (blue), server components (yellow), and common components (purple).

The server has a database-handler that provides operations to access the in-memory database, and a process-orchestrator that handles the server-side logic. database-handler's main responsibility is to persist data, it uses dog-profile's public operations to access profile data and does not need to know about the process-orchestrator. process-orchestrator depends on database-handler to store data and on dog-profile to create profile instances.

Name: Arne Lieten

Email: Arne.Lieten@vub.be

Student Number: 0642644

VRIJE UNIVERSITEIT BRUSSEL

Each client has a client-orchestrator that handles business logic and a main-window that manages the GUI. main-window handles opening and closing the window, manages the create-profile-screen, and will manage all other screens in later phases. The create-profile-screen is in charge of drawing the screen where dog profiles are created. Both the main-window and the create-profile-screen depend on the `racket/gui/base` library (not shown on diagram). The create-profile-screen passes submitted field values (raw-fields) to the client-orchestrator through a callback.

The client-orchestrator could have simply referenced the process-orchestrator. However, since in phase 2 we will work with TCP connection between the two a callback is used since this decouples the components more in favor of later phases of the project.

In later phases the main-window will manage multiple screens, this was the reason to split up the main-window and the create-profile-screen already. Furthermore the database-handler depends on the database interface to remove tight coupling with the in-memory-database and the sqlite database can be swapped in easily in phase 2.

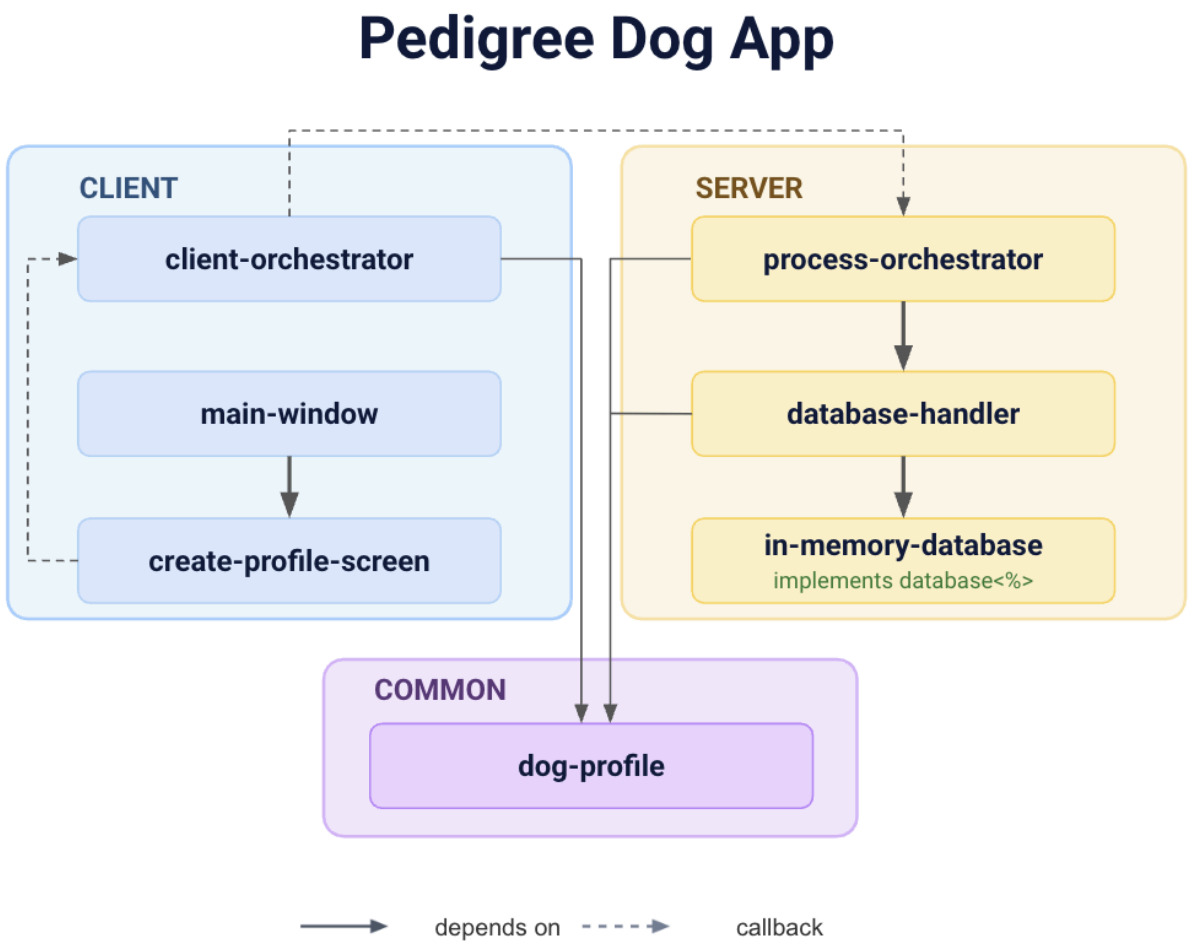


Image 1: ADT Dependency Diagram of Pedigree Dogs App

Name: Arne Lieten

Email: Arne.Lieten@vub.be

Student Number: 0642644

VRIJE UNIVERSITEIT BRUSSEL

Common ADTs

dog-profile

dog-profile represents information about a particular dog: id, name, age, hobbies and type of relation the dog is looking for. Table 1 gives the available operations and signatures. The client-orchestrator creates the dog-profile with the ID set to false, the ID gets populated by the process-orchestrator on the server side with help of the `uuid` library.

Operation	Signature
initialize-dog-profile	ID, string, integer, string, string -> dog-profile
dog-profile-id	dog-profile -> ID
dog-profile-name	dog-profile -> string
dog-profile-age	dog-profile -> integer
dog-profile-hobbies	dog-profile -> string
dog-profile-relation	dog-profile -> string

Table 1: ADT of the dog-profile

Server ADTs

in-memory-database

Stores the dog profiles in a hash table, with the ID as key and the profile fields as value. It implements the database interface used by the database-handler, so it can be replaced by the SQLite database in phase 2. The read and update! operations raise an exception if the ID does not exist. The operations and signatures of the in-memory-database are seen in Table 2.

Operation	Signature
create!	in-memory-database, ID, hash -> void
read	in-memory-database, ID -> hash
update!	in-memory-database, ID, hash -> void
delete!	in-memory-database, ID -> void

Table 2: ADT of the in-memory-database

Name: Arne Lieten

Email: Arne.Lieten@vub.be

Student Number: 0642644

VRIJE UNIVERSITEIT BRUSSEL

database-handler

ADT responsible for persisting data to the in-memory-database in phase 1 and the SQLiteDB in phase 2. Next to that the database-handler can perform CRUD operations on the database to create, read, update and delete dog-profiles. If the read-profile or update-profile! operations cannot find an object with the ID it will raise an exception. The operations and corresponding signatures for the database-handler are displayed in Table 3.

Operation	Signature
create-profile!	database-handler, dog-profile -> void
read-profile	database-handler, ID -> dog-profile
update-profile!	database-handler, dog-profile -> void
delete-profile!	database-handler, ID -> void

Table 3: ADT of the database-handler

process-orchestrator

Coordinates profile creation for all clients and holds a reference to the database-handler. It receives a dog-profile with #f as ID and generates a UUID for the ID based on the `uuid` library. The complete dog-profile is created during the create-dog-profile! operation and sent to the database-handler to be stored. The signatures and operations for the process-orchestrator are seen in Table 4.

Operation	Signature
create-dog-profile!	process-orchestrator, dog-profile -> symbol

Table 4: ADT of the process-orchestrator

Client ADTs

client-orchestrator

Handles main business logic for the client. Information gathered from the GUI is passed on via callback from the create-profile-screen. The client-orchestrator uses a callback to communicate with the shared process-orchestrator in phase 1. In phase 2 when communication will go over TCP the callback can be updated easily compared to if we had worked with references.

The operation submit-dog-profile! validates the raw-fields from the create-profile-screen. It returns 'incomplete-profile if any field is empty and 'invalid-age if the age cannot be converted to a number. If the raw-fields are complete and valid, it creates a dog-profile with #f as ID and passes it to the process-orchestrator. The operations and signatures of the client-orchestrator are seen in Table 5.

Name: Arne Lieten
Email: Arne.Lieten@vub.be
Student Number: 0642644

Operation	Signature
submit-dog-profile!	client-orchestrator, raw-fields -> symbol

Table 5: ADT of the client-orchestrator

main-window

Responsible for one client GUI, main-window creates the window and manages the different screens that can be shown in it. In phase 1 only the create-profile-screen can be shown, in later phases this will be extended to other screens (swipe-screen, login-screen, …). It receives the submit callback at creation and passes it on to the create-profile-screen, so main-window never calls it itself and does not need to know about the client-orchestrator. main-window provides operations to open and close the window and to switch between screens. The main-window's operations and signatures are found in Table 6.

Operation	Signature
open-window!	main-window -> void
close-window!	main-window -> void
show-screen!	main-window, symbol -> void

Table 6: ADT of the main-window

create-profile-screen

Shows the form to create a dog profile: text fields for name, age, hobbies and relation, and a submit button and a feedback message. It receives the submit callback at creation. When the submit button is clicked, it collects the field values as raw-fields, passes them to the callback and translates the returned symbol into a feedback message ('saved, 'incomplete-profile or 'invalid-age). The screen only handles presentation, and all checks happen in the client-orchestrator. The create-profile-screen has no public operations and is created by main-window.

main

Main wires together all components and starts the application with four GUIs and one server. Main first creates the server: an in-memory-database, a database-handler and a process-orchestrator. For each client, it then creates a client-orchestrator and a main-window.

Main also sets up the callback functions between the client-orchestrator and the process-orchestrator to forward the dog-profile, and between the create-profile-screen and the client-orchestrator, via the main-window, to pass on the raw-fields.

Name: Arne Lieten

Email: Arne.Lieten@vub.be

Student Number: 0642644

VRIJE UNIVERSITEIT BRUSSEL

Testing

All the files have their own unit tests except for main-window and create-profile-screen since they touch the GUI, main is also not tested. To keep the unit tests structured the folder structure mimics the root structure of the project. Mock objects are used in the tests of the database-handler, the process-orchestrator, and the client-orchestrator. The reason for this was so that each ADT can be tested independently of the others. Even though there are no unit tests for the files that touch the GUI, these are tested manually in the GUI to have good coverage of the codebase.